PROJECT EVIDENCE & WALKTHROUGH

Enterprise-Grade CI/CD Pipeline on AWS ECS Fargate

From git push to production, fully automated:
GitHub Actions → Docker → Amazon ECR → ECS Fargate, provisioned with Terraform

GitHub — Actions — Docker — ECR — ECS — ALB

Yemi Abiona · October 2026

38
AWS resources provisioned by Terraform

4m 27s
push-to-production, verified automatically

0
long-lived AWS keys stored in GitHub

2 / 2
healthy load balancer targets after deploy

What this document shows

A small Node.js service deployed through a production-style delivery pipeline: every push to main is tested, containerised, pushed to Amazon ECR and rolled out to ECS Fargate behind an Application Load Balancer. The pages that follow walk through the build with screenshots, including the problems met along the way and how each was diagnosed and fixed.

Terraform · GitHub Actions · Docker · AWS ECS Fargate · Amazon ECR · ALB · IAM OIDC · CloudWatch · Node.js

github.com/yemabion/ECS-Fargate-CICD-Pipeline

Contents

1. Architecture & pipeline design . . . 3

2. Application & container . . . 5

3. Infrastructure as code with Terraform . . . 6

4. The running service on ECS Fargate . . . 9

5. Connecting GitHub Actions to AWS . . . 10

6. Troubleshooting log . . . 12

7. Pipeline success & proof of deployment . . . 16

8. Lessons learned & next steps . . . 20

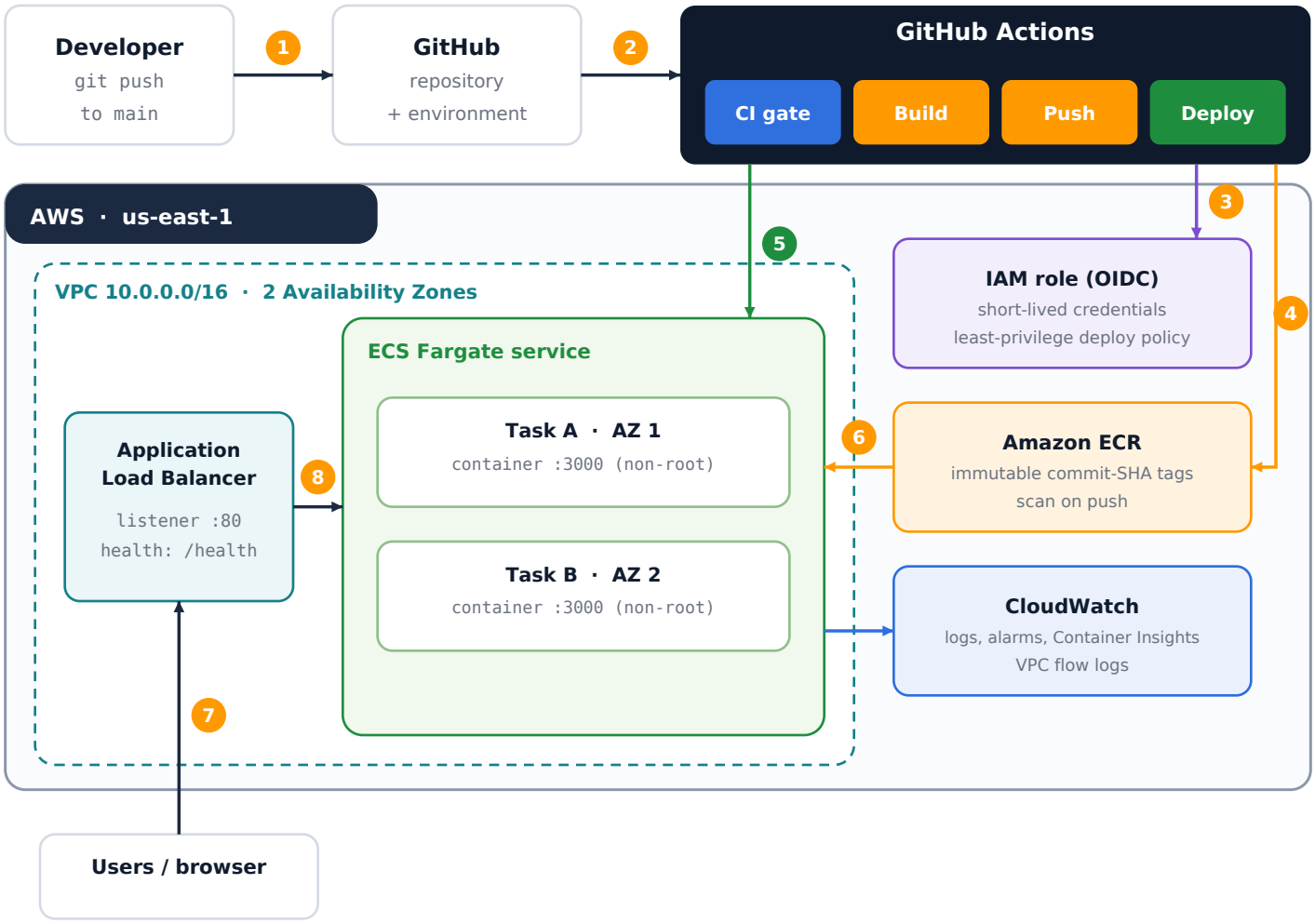
Project at a glance

A small Node.js service that replies **Hello from CI/CD Pipeline!** is the payload for a production-style delivery pipeline on AWS. Infrastructure is defined entirely in **Terraform**, images are built and shipped by **GitHub Actions**, and the service runs on **ECS Fargate** behind an **Application Load Balancer**.

Area	What was built
Authentication	GitHub OIDC → short-lived AWS credentials. The role trusts only this repository and the `production` environment; no access keys are stored in GitHub.
Supply chain	Immutable ECR tags (tag = commit SHA), scan on push, lifecycle policy, Dependabot for Actions, Docker, npm and Terraform.
Reliability	Two Availability Zones, rolling deployments, circuit-breaker auto-rollback, CPU-based autoscaling, and a pipeline check that the new deployment actually reached `COMPLETED`.
Security	Non-root container with read-only root filesystem, tasks reachable only from the ALB security group, locked default security group, VPC flow logs, encrypted and versioned Terraform state.
Observability	Structured JSON logs in CloudWatch, Container Insights, alarms for unhealthy targets and 5xx errors.
Delivery	Quality gate (unit tests, container smoke test, `terraform fmt/validate`) before any deploy; one deployment at a time; remote state with locking.

1 Architecture & pipeline design

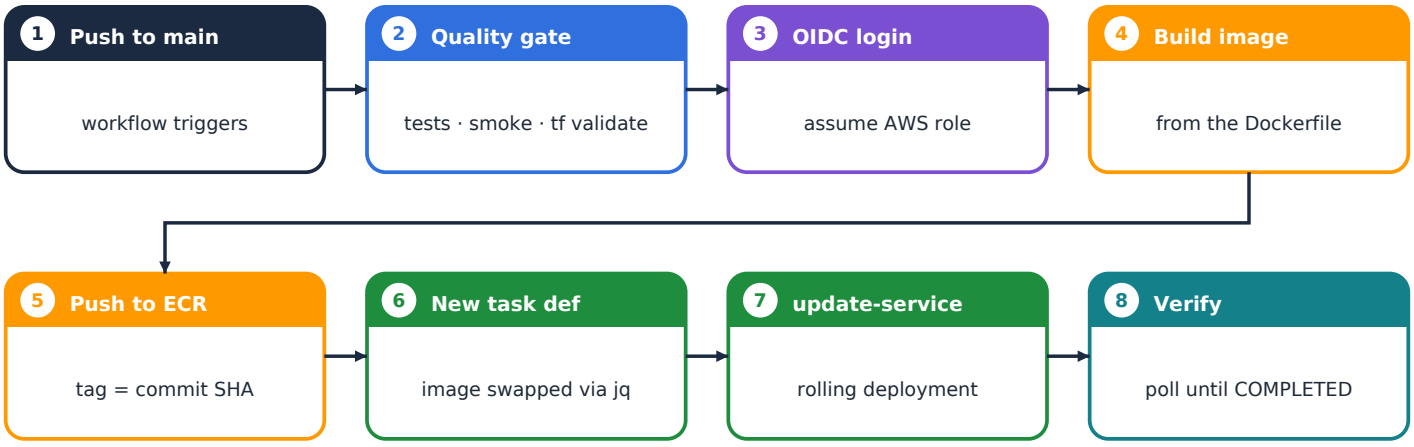
How the pieces fit together



1. Developer pushes to `main`.
2. GitHub triggers the workflow.
3. The job requests an OIDC token and assumes a short-lived AWS role.
4. Image is pushed to ECR, tagged with the commit SHA.
5. A new task-definition revision is registered and the ECS service updated.
6. Fargate pulls the image from ECR.
7. Users reach the application through the load balancer.
8. The ALB forwards to healthy tasks on port 3000.

The deployment pipeline

Eight stages from commit to verified release. Nothing reaches ECR or ECS unless the quality gate passes.



Automatic rollback: the ECS deployment circuit breaker reverts a release that fails to become healthy.

Deliverables checklist

Each requirement of the project and where the evidence is in this document.

	Requirement	Evidence
✔	Web app returns **Hello from CI/CD Pipeline!**	Fig. 1, Fig. 13
✔	Dockerfile builds and the container serves the app	Fig. 2 · container probed in CI Fig. 24
✔	Terraform: VPC, subnets, internet gateway, security groups, ECR, ECS cluster, task definition, service, ALB	Fig. 3
✔	ECR repository containing the image	Fig. 4, Fig. 8, Fig. 28
✔	ECS cluster with running tasks	Fig. 10
✔	Load balancer targets healthy	Fig. 11
✔	GitHub Actions: checkout, build, ECR auth via `configure-aws-credentials`, push, `aws ecs update-service`, trigger on push to `main`	Fig. 24, Fig. 25
✔	Deployed app reachable through the load balancer DNS name	Fig. 13, Fig. 26
✔	Repository with app, Dockerfile, `/infra`, `/.github/workflows` and README	Fig. 33

2 Application & container

Built and verified locally first

The service is a dependency-free Node.js HTTP server with a `/health` endpoint, graceful shutdown and structured logs. It has six unit tests, and the Dockerfile produces a small non-root image.

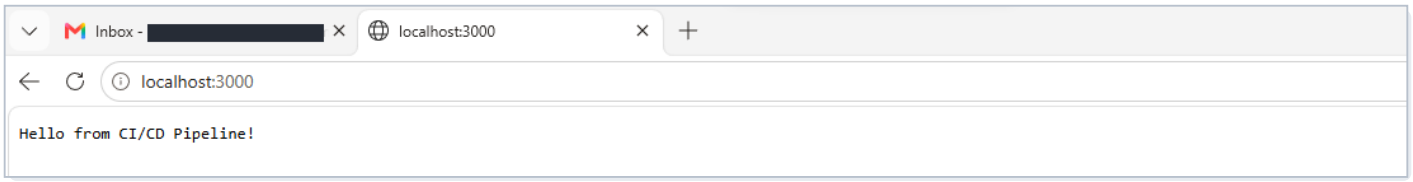


Figure 1. The service answering on `localhost:3000` with the required response, **Hello from CI/CD Pipeline!**

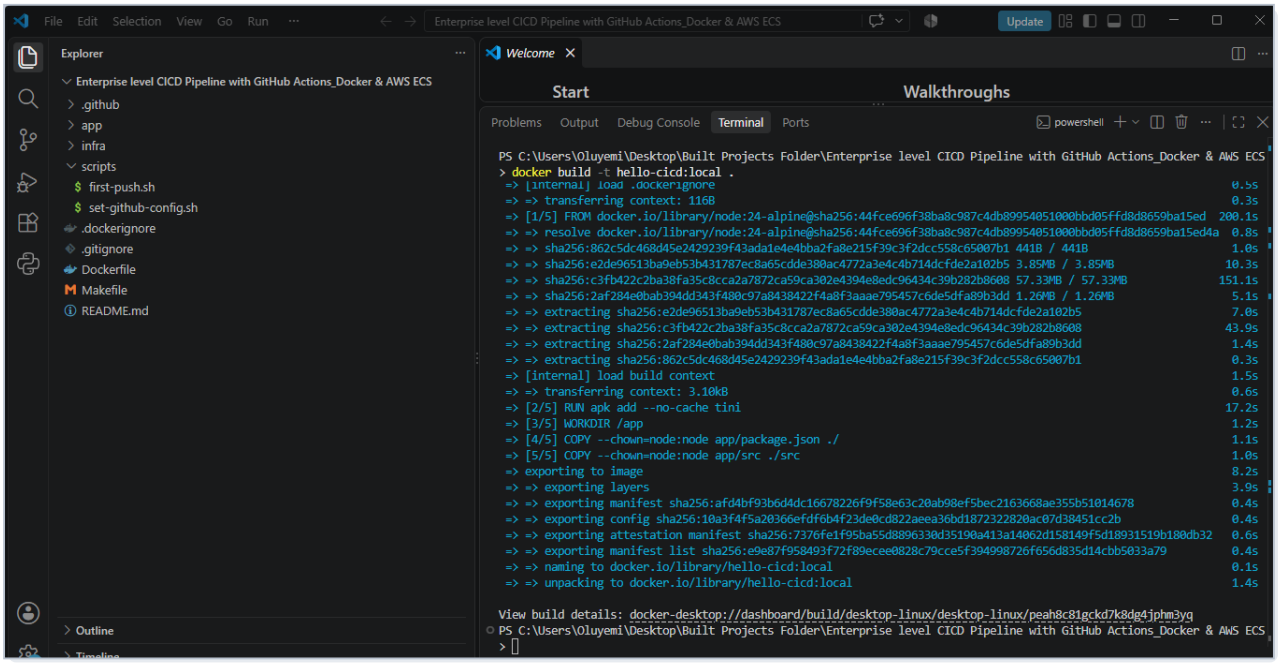


Figure 2. `docker build` completing successfully: a slim `node:24-alpine` image that runs as a non-root user and includes a health check.

3 Infrastructure as code with Terraform

Network, registry, cluster, load balancer, IAM

Everything in AWS is defined in Terraform with remote state in S3. The ECR repository is created first with a targeted apply so the ECS service has an image to start from; the remaining 37 resources follow in a single apply (Fig. 3). The figures after it show the repository and the first image pushed to it.

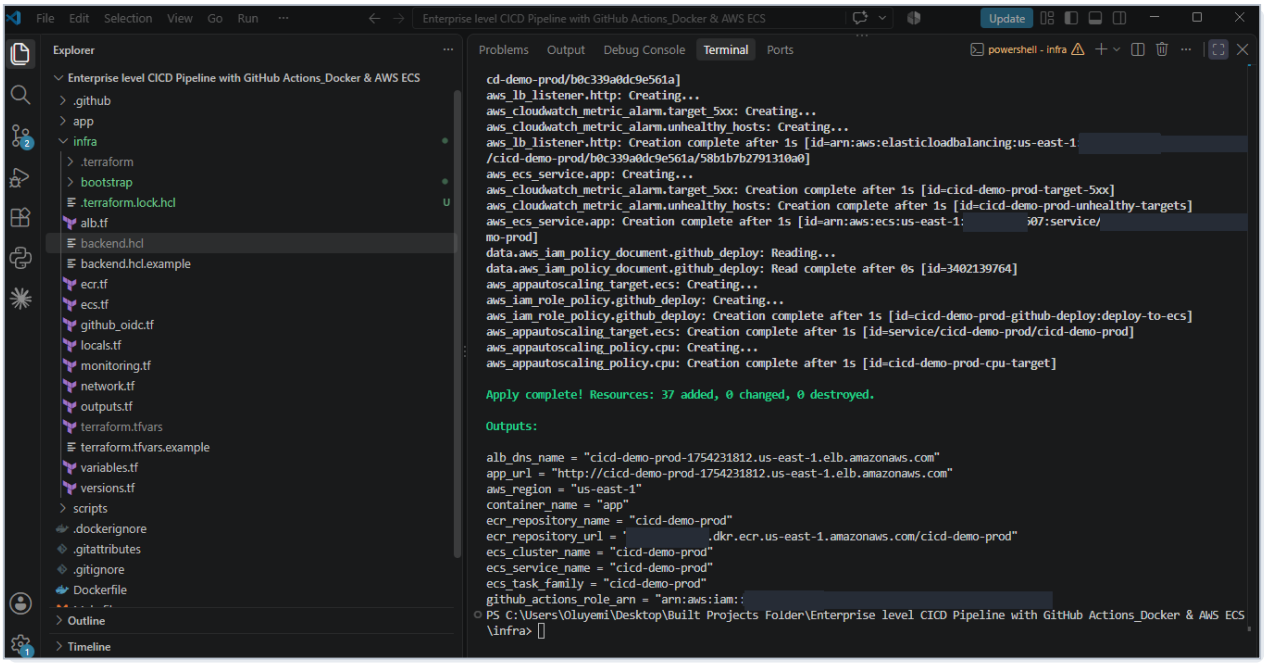


Figure 3. `terraform apply`: **37 resources created** (network, security groups, ALB, ECS, IAM, alarms) on top of the ECR repository from the earlier targeted apply, 38 in total. The outputs include the load balancer DNS name.

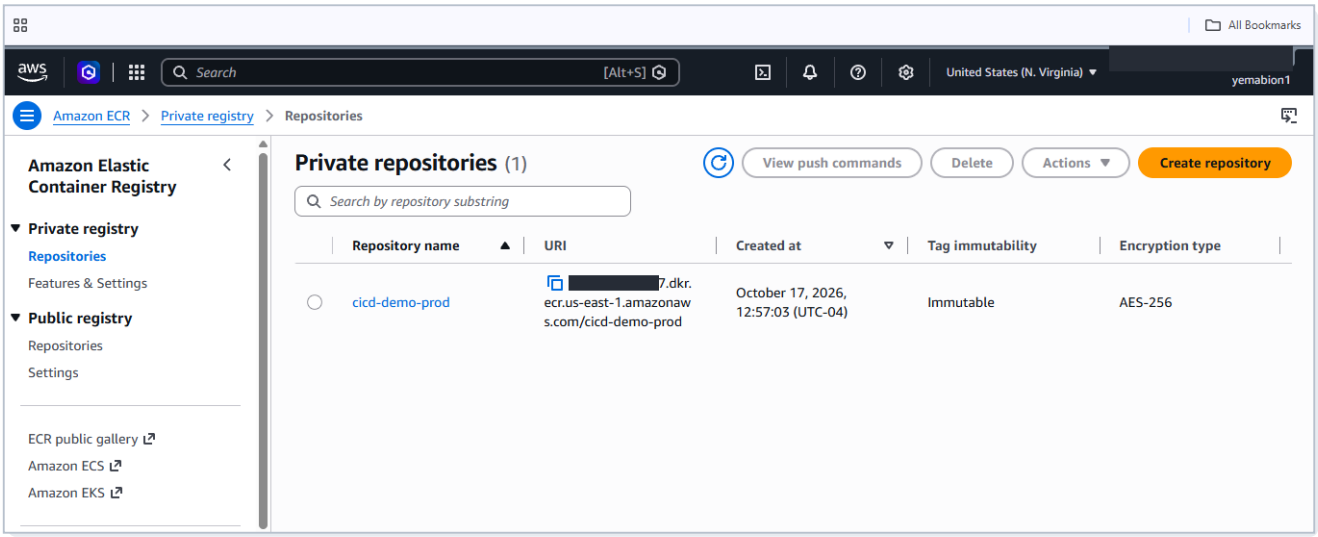


Figure 4. Private ECR repository `cicd-demo-prod`, created by Terraform with **immutable tags** and AES-256 encryption.

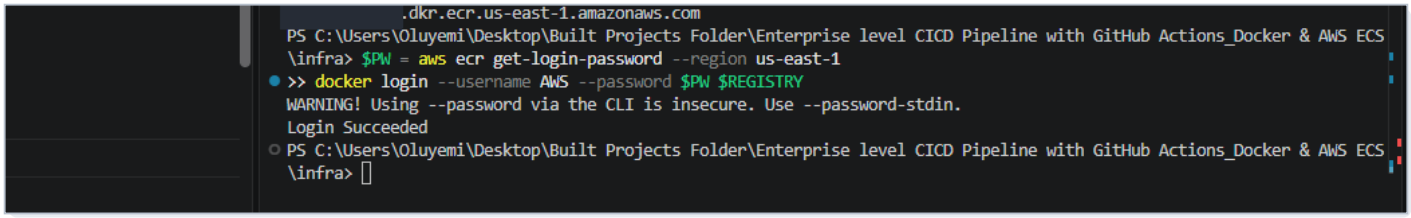


Figure 5. Authenticating Docker to Amazon ECR: **Login Succeeded**.

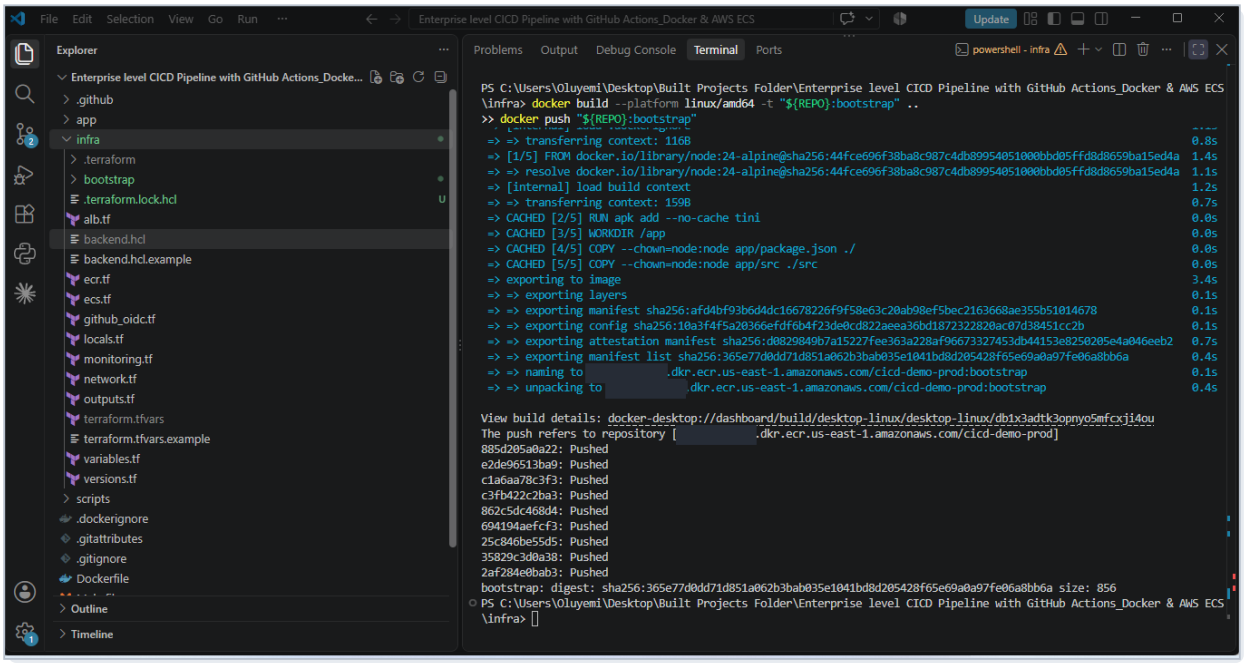


Figure 6. Building for `linux/amd64` (the Fargate CPU type) and pushing the first image, tagged `bootstrap`. The push ends with the image digest.

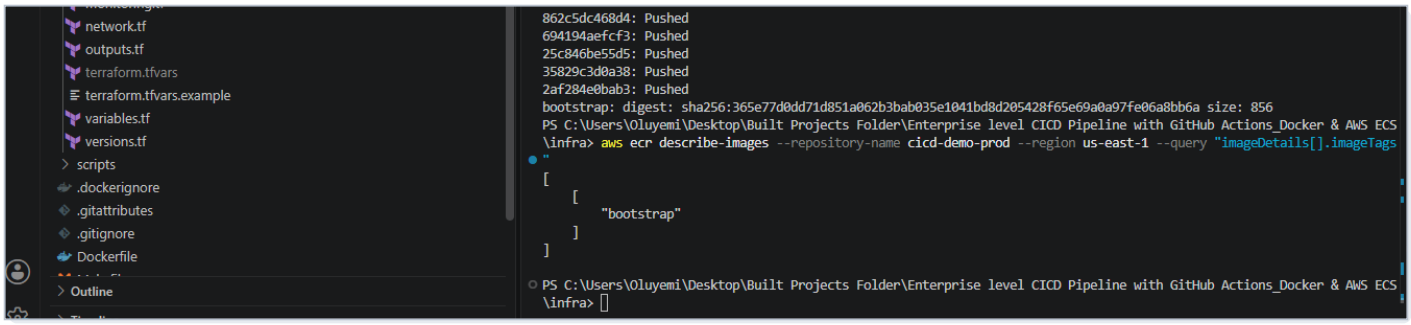


Figure 7. Confirming the pushed tag from the CLI with `aws ecr describe-images`.

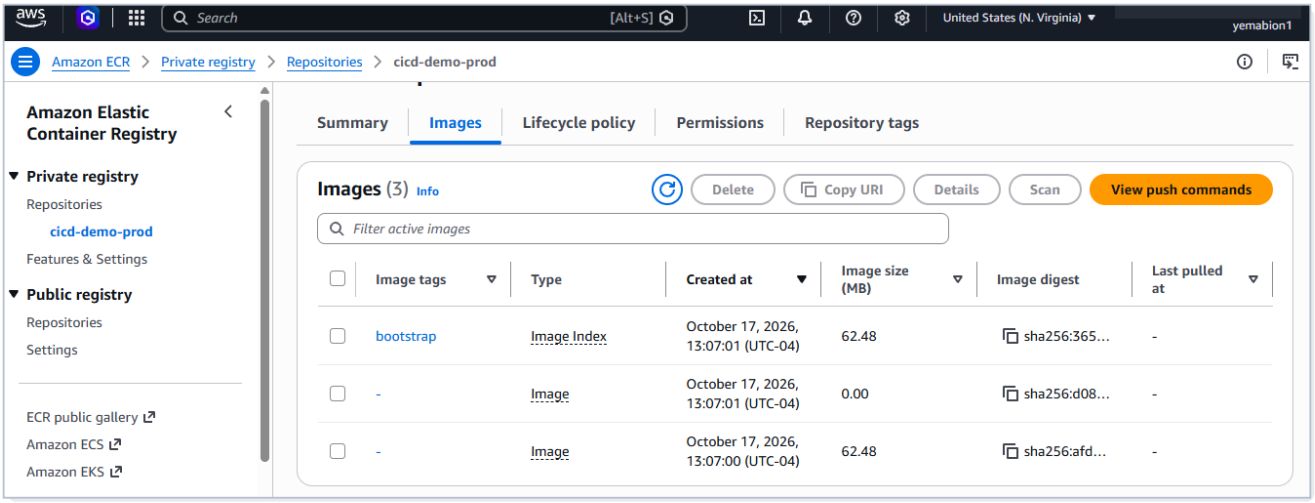


Figure 8. The image visible in the ECR console (an image index plus its platform manifests).

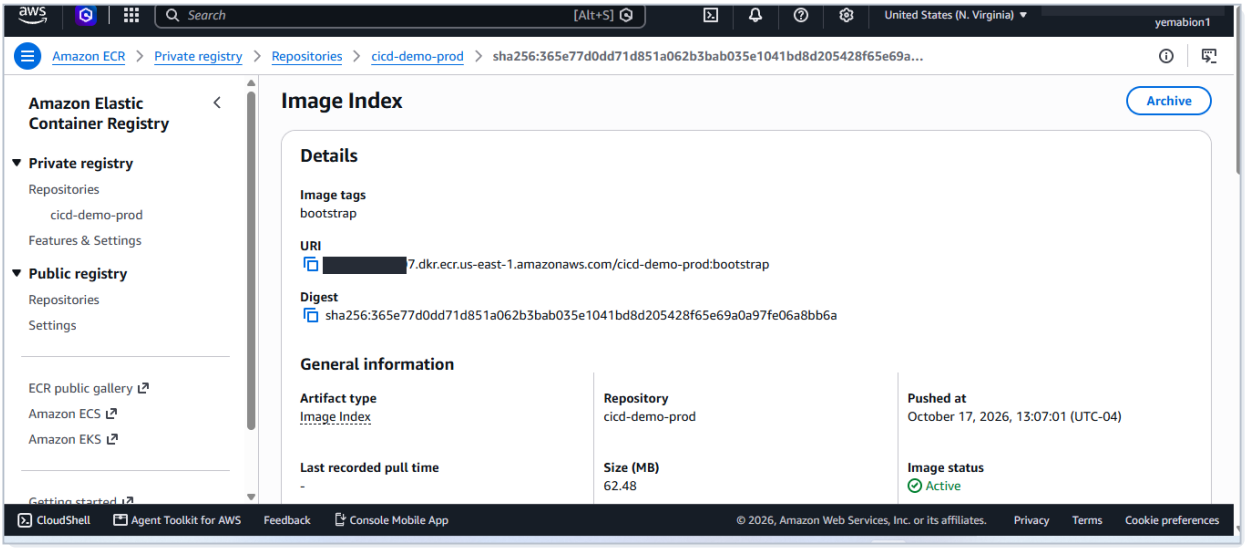


Figure 9. Image details: tag, URI, digest, size, and an **Active** status.

4 The running service on ECS Fargate

Tasks, target health and the public endpoint

Two Fargate tasks run behind the Application Load Balancer. Both are healthy in ECS and in the target group, and the public DNS name serves the application.

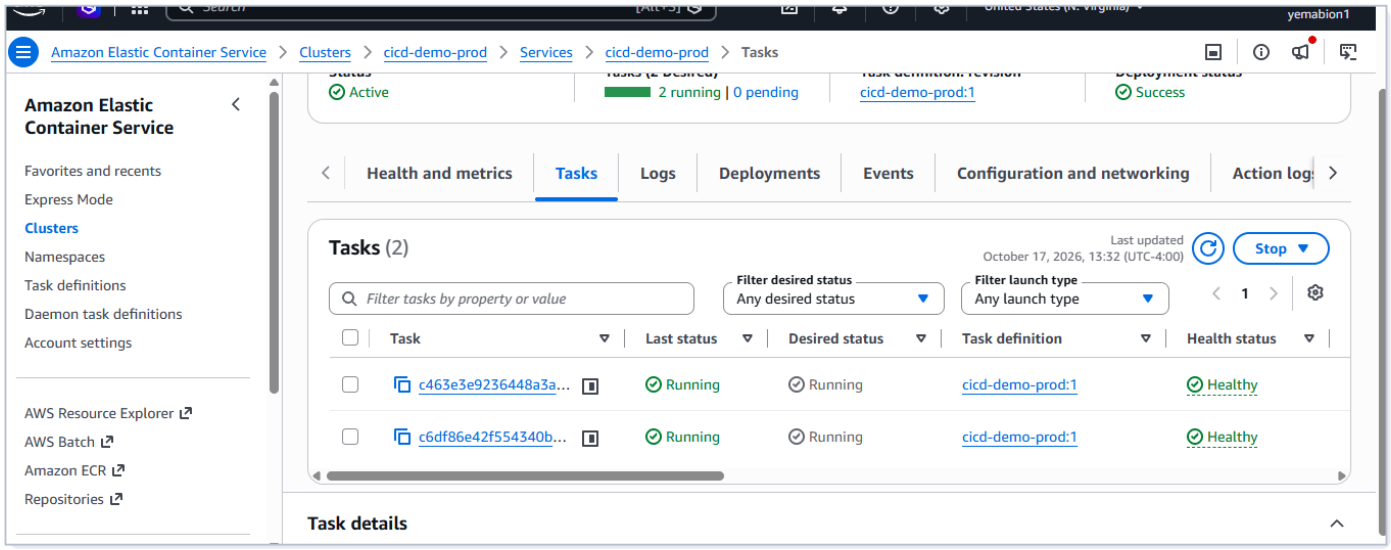


Figure 10. ECS service `cicd-demo-prod`: **2 tasks Running**, both reporting **Healthy**.

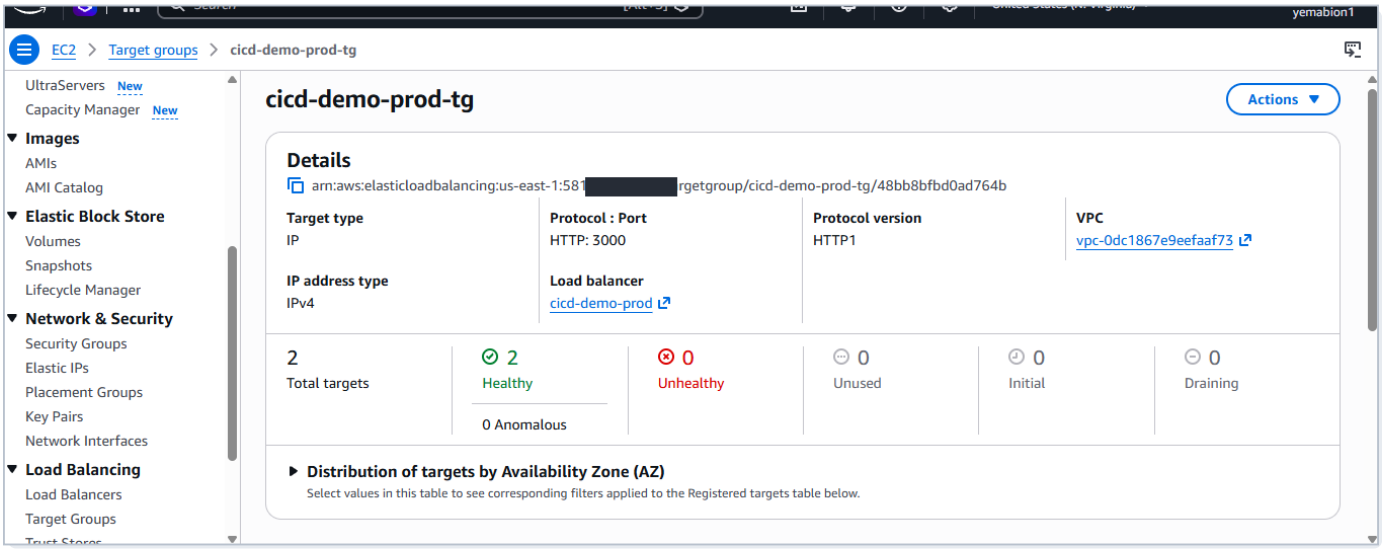


Figure 11. ALB target group `cicd-demo-prod-tg`: 2 registered targets, **2 Healthy, 0 Unhealthy**.

```
PS C:\Users\Oluyemi\Desktop\Built Projects Folder\Enterprise level CICD Pipeline with GitHub Actions_Docker & AWS ECS
\infra> aws ecs describe-services --cluster cicd-demo-prod --services cicd-demo-prod --region us-east-1 --query "serv
ices[0].[runningCount,desiredCount]"
[
    2,
    2
]

PS C:\Users\Oluyemi\Desktop\Built Projects Folder\Enterprise level CICD Pipeline with GitHub Actions_Docker & AWS ECS
\infra>
```

```
PS C:\Users\Oluyemi\Desktop\Built Projects Folder\Enterprise level CICD Pipeline with GitHub Actions_Docker & AWS ECS
\infra> curl.exe (terraform output -raw app_url)
Hello from CI/CD Pipeline!
PS C:\Users\Oluyemi\Desktop\Built Projects Folder\Enterprise level CICD Pipeline with GitHub Actions_Docker & AWS ECS
\infra>
```

Figure 12. Desired vs running task count (2 / 2), then a `curl` request through the load balancer returns the expected response.



Figure 13. The deployed application in a browser, reached through the **load balancer DNS name**.

5 Connecting GitHub Actions to AWS

Reproducible providers and configuration

The workflow reads its settings from repository *variables* (not secrets) and authenticates with OIDC, so there are no credentials to rotate or leak.

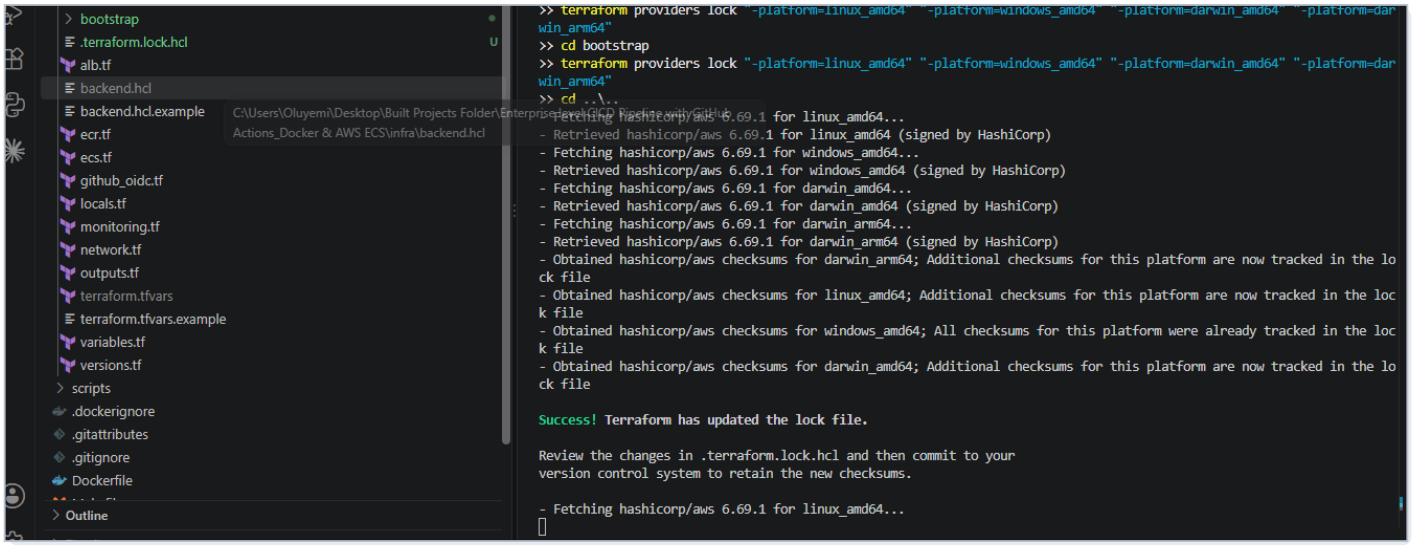


Figure 14. `terraform providers lock` for Linux, Windows and macOS, so local machines and the Linux CI runners resolve identical provider builds.

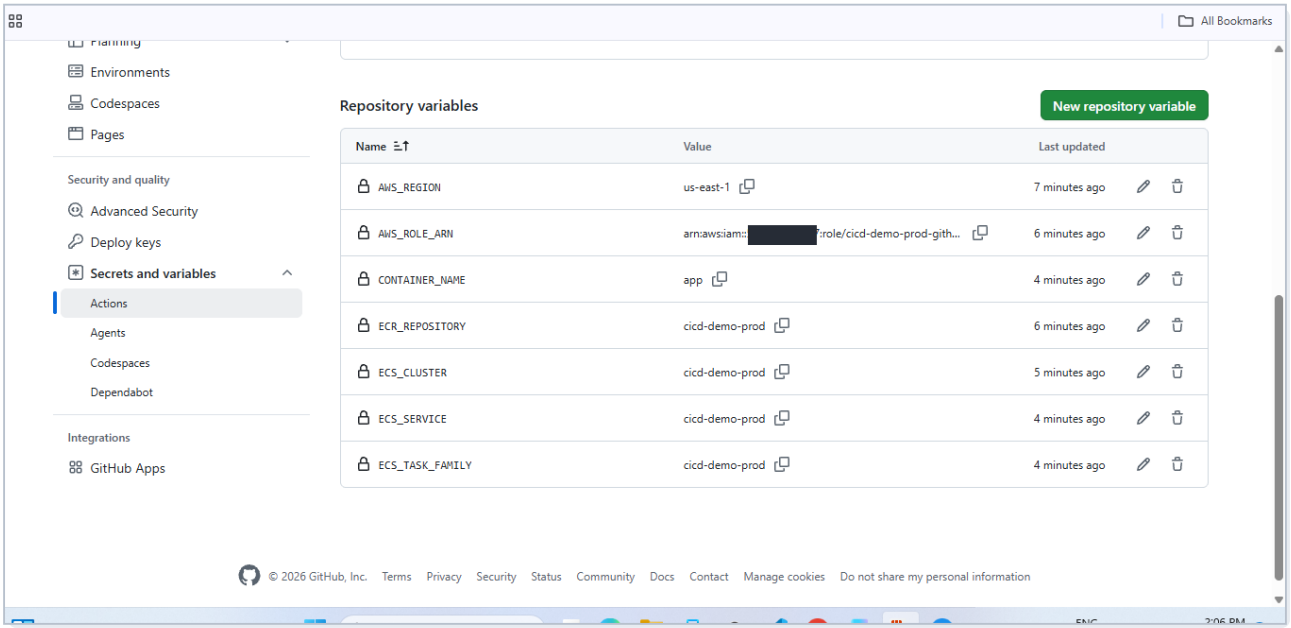


Figure 15. Seven GitHub Actions **repository variables**: names and identifiers only; no access keys or secrets.

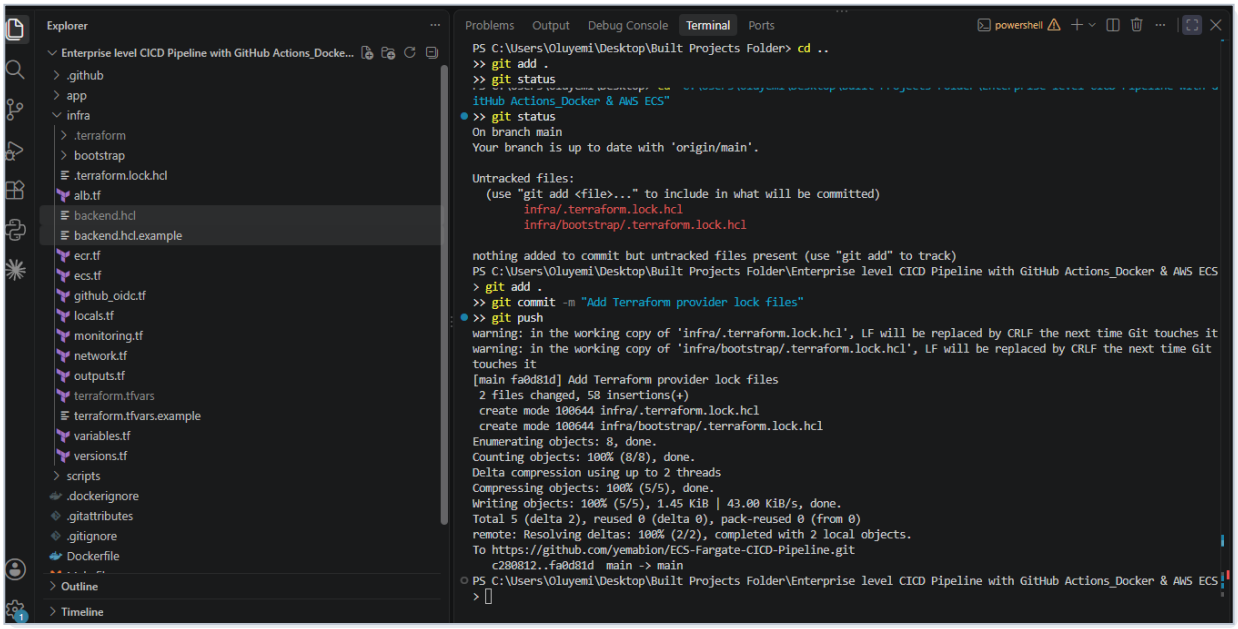


Figure 16. Committing the provider lock files and pushing to `main`, the push that starts the pipeline.

6 Troubleshooting log

Three real problems, each diagnosed with evidence

Issue 1: AWS rejected the GitHub OIDC login

Symptom	The deploy job failed at *Configure AWS credentials* with `Not authorized to perform sts:AssumeRoleWithWebIdentity` (Fig. 17).
Root cause	The IAM trust policy expected the classic sub format, but GitHub now includes numeric owner and repository IDs for newer repositories (Fig. 18).
Fix	Allow both formats in the Terraform trust policy, then apply (Fig. 19). The run goes green (Fig. 20).

```
GitHub sent : repo:yemabion@283285674/ECS-Fargate-CICD-Pipeline@1420817404:environment:production
AWS expected: repo:yemabion/ECS-Fargate-CICD-Pipeline:environment:production
```

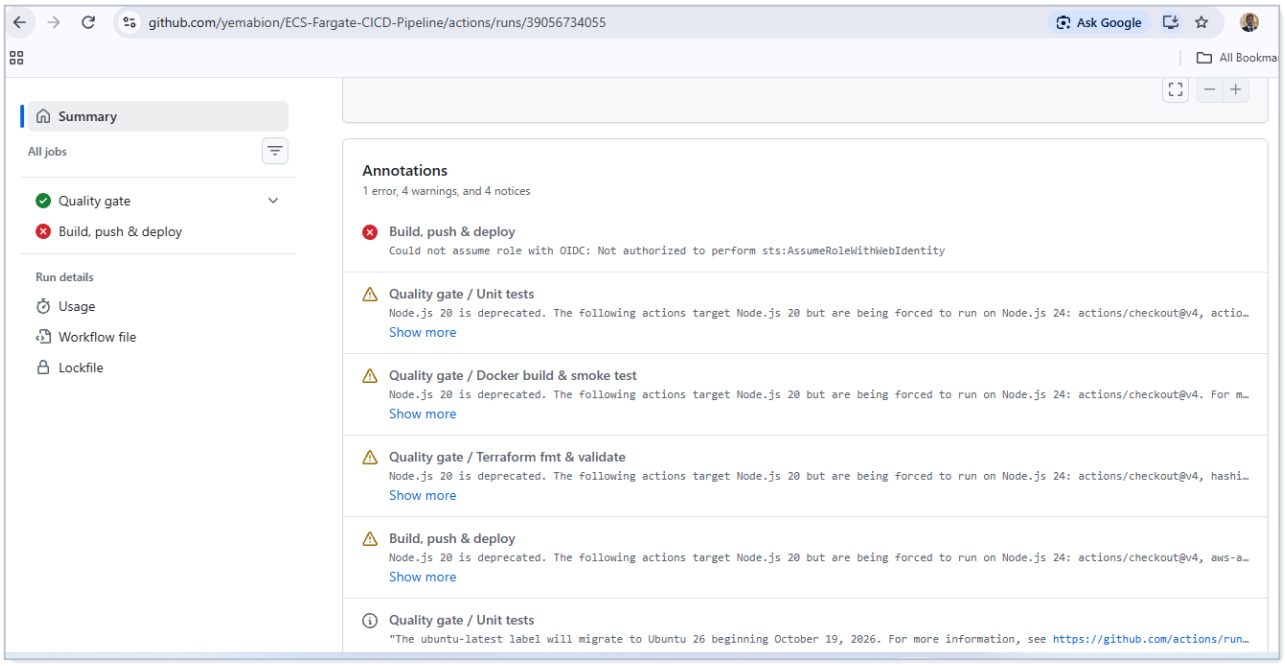


Figure 17. First deploy attempt: the job fails at *Configure AWS credentials* with `Not authorized to perform sts:AssumeRoleWithWebIdentity`.

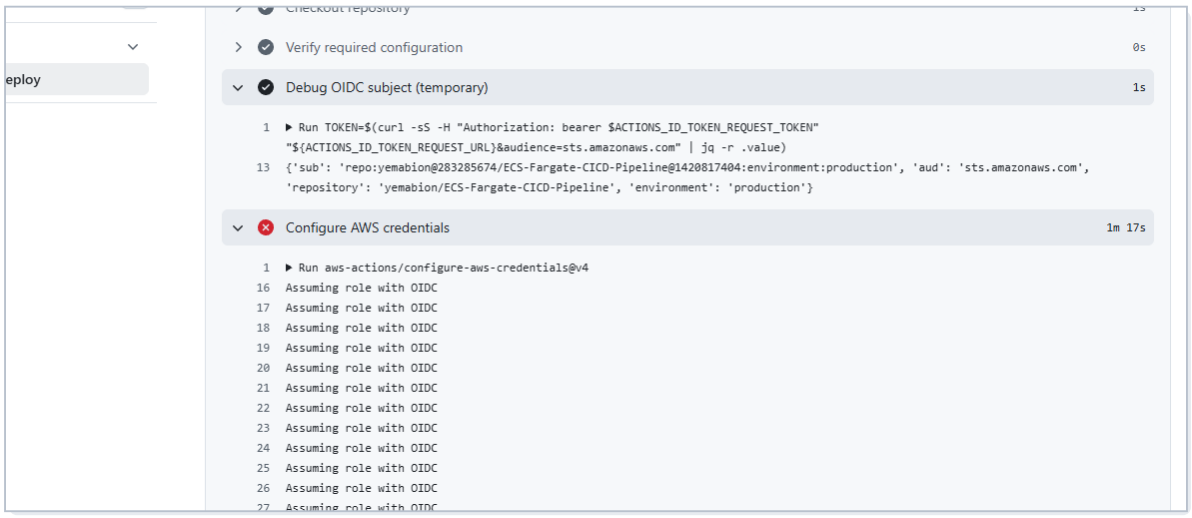


Figure 18. A temporary debug step prints the claims in GitHub's OIDC token. The sub claim contains numeric owner and repository IDs.

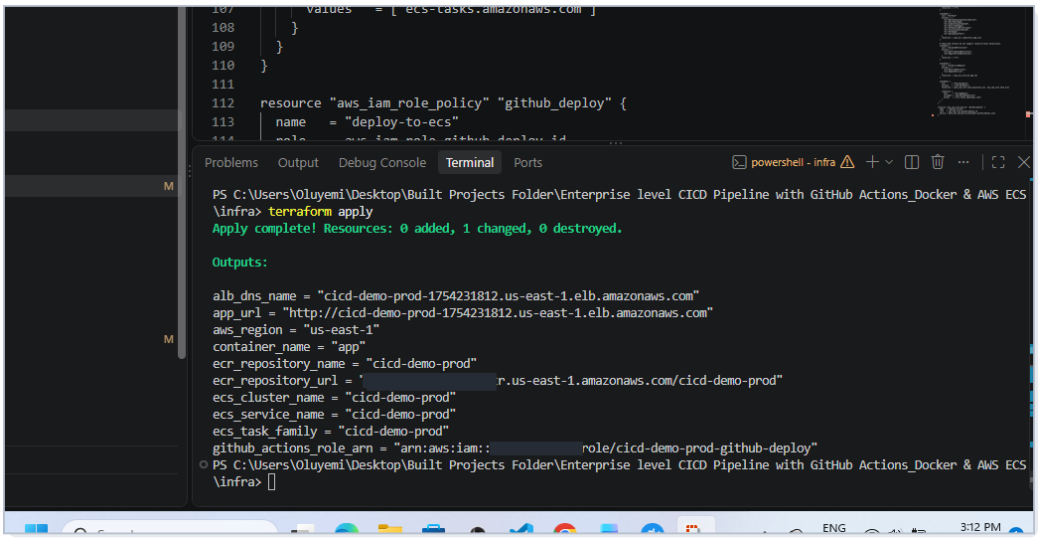


Figure 19. The fix: `terraform apply` changes only the IAM role's trust policy (**0 added, 1 changed, 0 destroyed**).

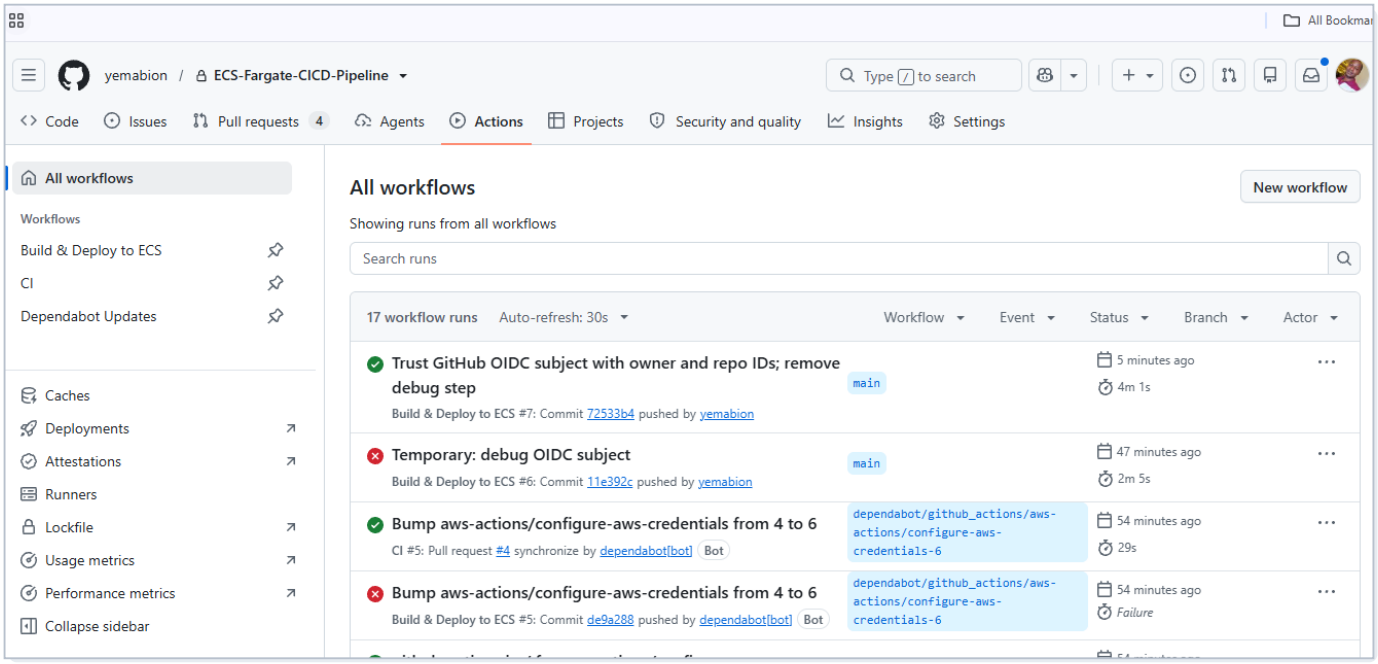


Figure 20. Run history after the fix: the pipeline goes from red to green.

Issue 2: the registry refused to overwrite a released image

Symptom	A manual re-run of an already deployed commit failed at *Push image to ECR* (Fig. 21).
Root cause	ECR tags are **immutable** and the pipeline tags each image with the commit SHA, so the same tag cannot be pushed twice. This is deliberate: a released tag can never be silently replaced.
Fix	No code change needed. Push a new commit to release again, or roll back by pointing the service at an earlier task definition.

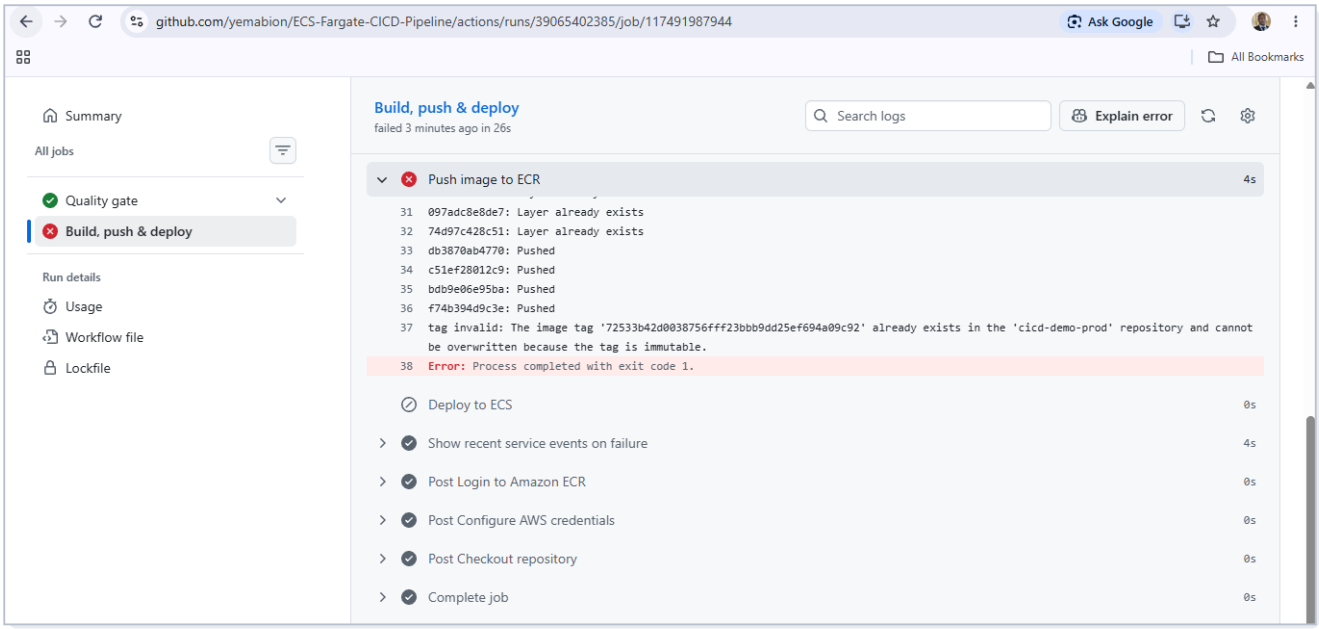


Figure 21. Re-running a commit that was already deployed is rejected: ECR tag immutability refuses to overwrite the released image.

Issue 3: the deploy check raced ECS's own state change

Symptom	A healthy deployment was reported as failed: `Rollout state: IN_PROGRESS` (Fig. 22).
Root cause	`aws ecs wait services-stable` returned slightly before ECS flipped the deployment to **COMPLETED** (Fig. 23). The pipeline checked the state exactly once.
Fix	Poll the rollout state every 10 seconds (up to 10 minutes) until it is **COMPLETED**, and fail immediately on **FAILED** (an automatic rollback).

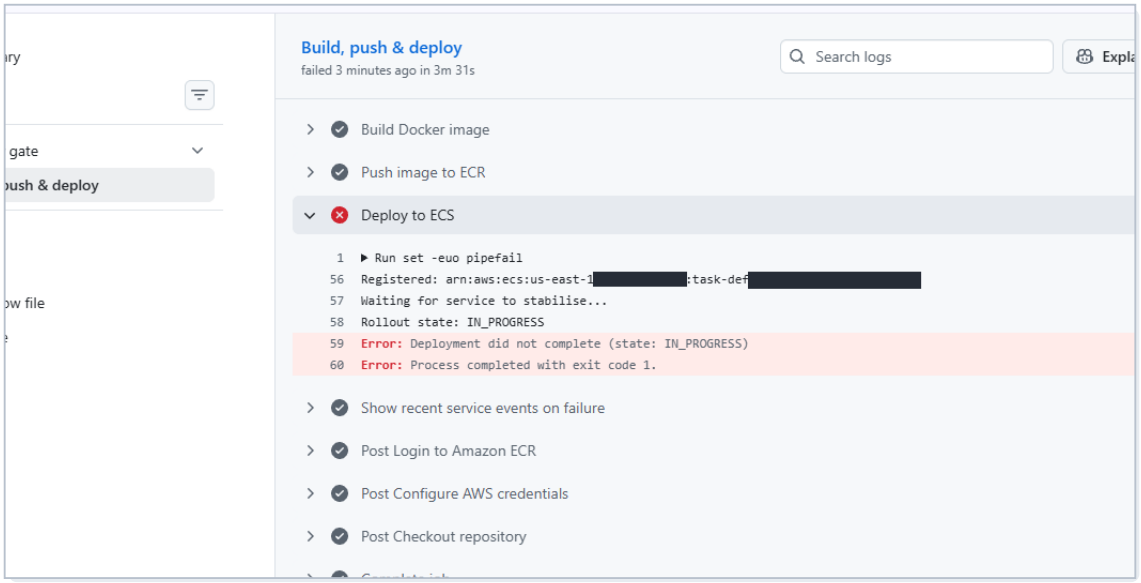


Figure 22. The deploy step read the rollout state once, while ECS still reported **IN_PROGRESS**, and failed a healthy deployment.

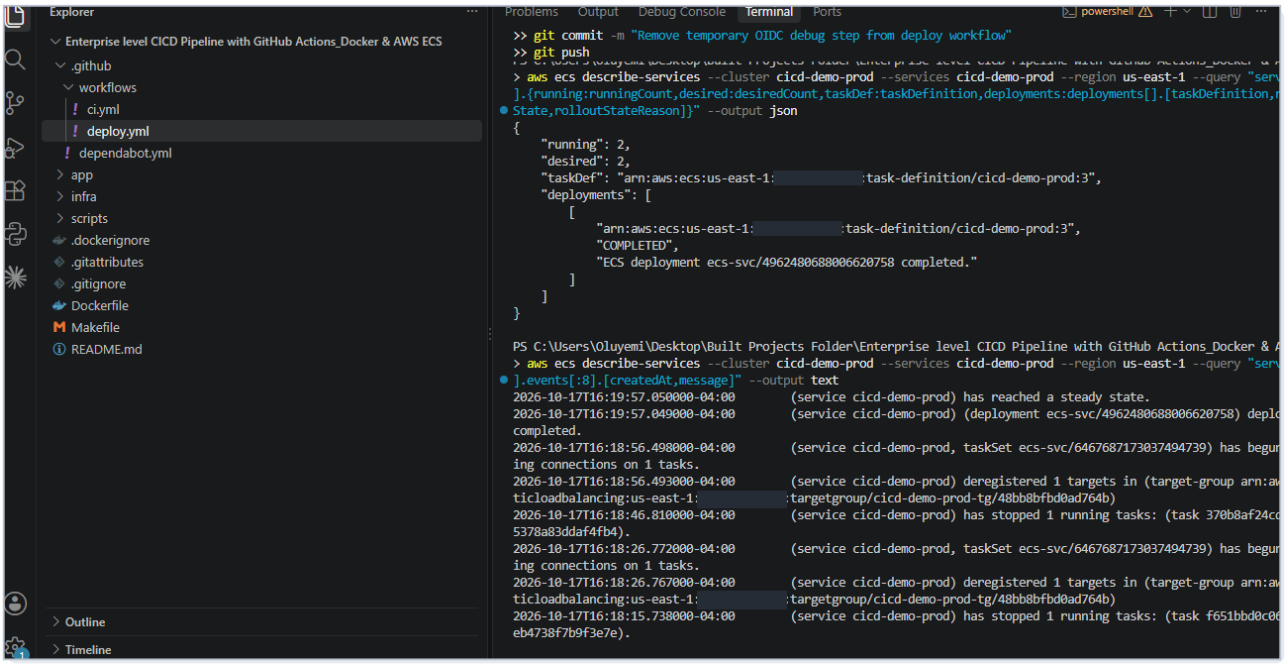


Figure 23. A minute later the same service shows deployment **COMPLETED** and *has reached a steady state* (2 running, task definition :3).

7 Pipeline success & proof of deployment

From commit to running container

The final run passes every stage. The image running in production carries the commit SHA, so any running container can be traced back to the exact commit that built it.

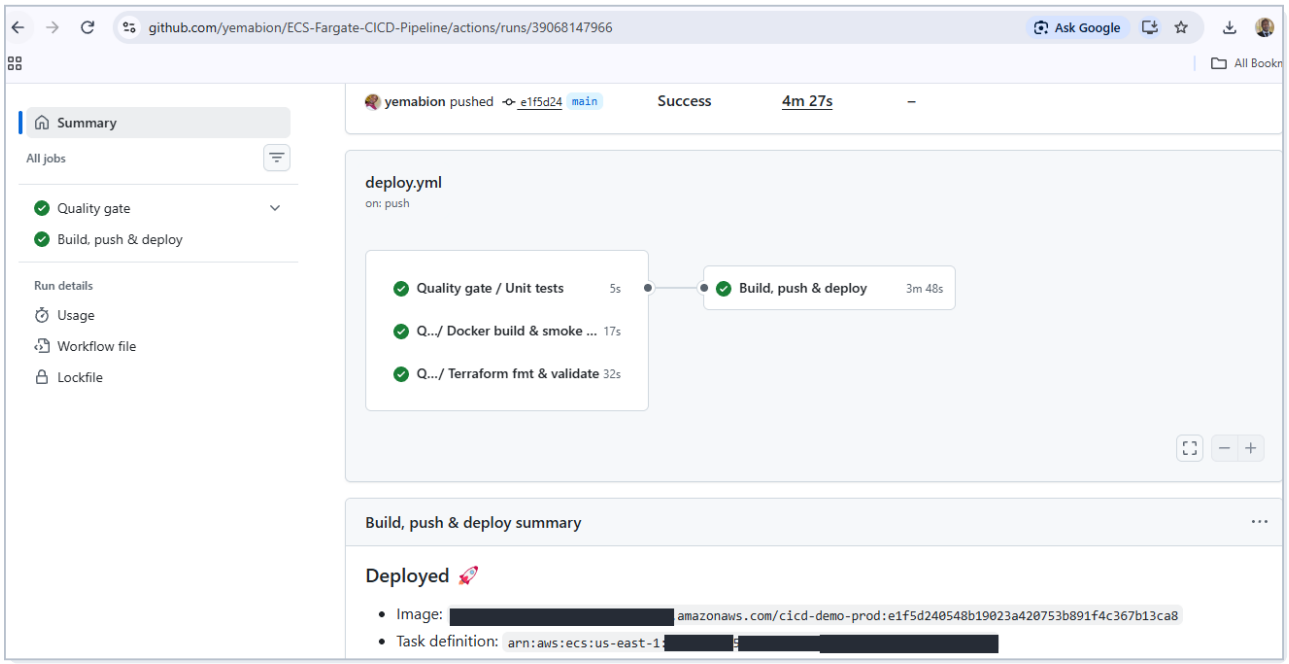


Figure 24. Final pipeline run: the quality gate (unit tests, Docker smoke test, Terraform validate) and the build-push-deploy job are all green. The summary lists the commit-SHA image and task definition **:4**.

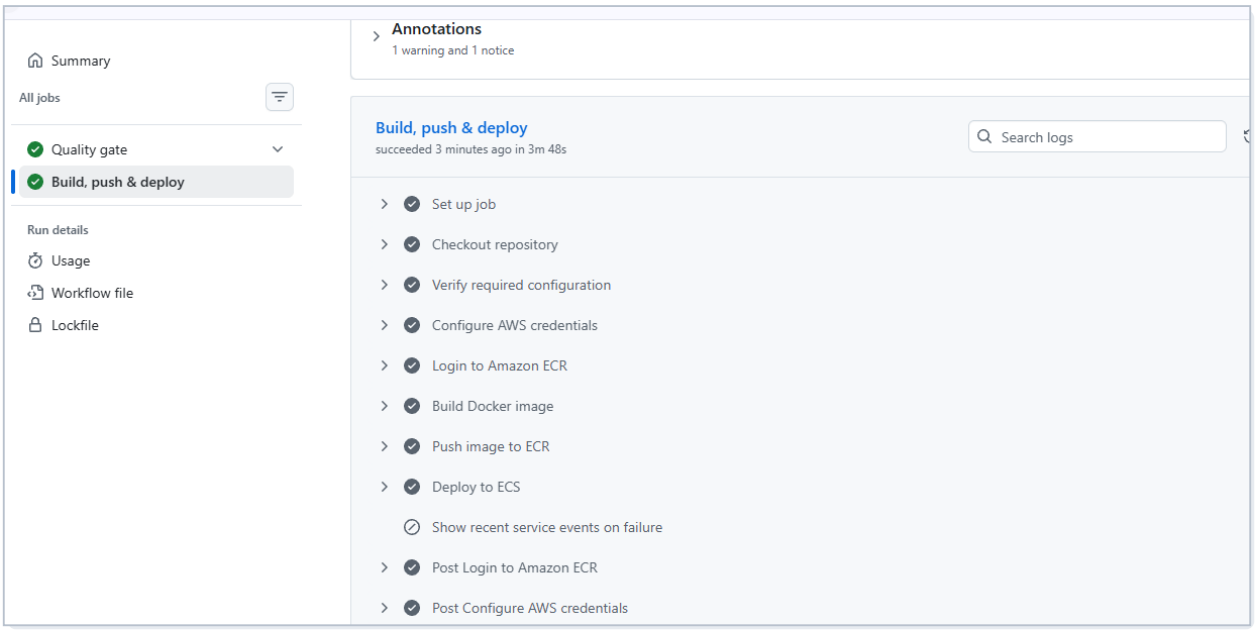


Figure 25. Every step of the deploy job passes: OIDC authentication, ECR login, build, push and ECS deploy.

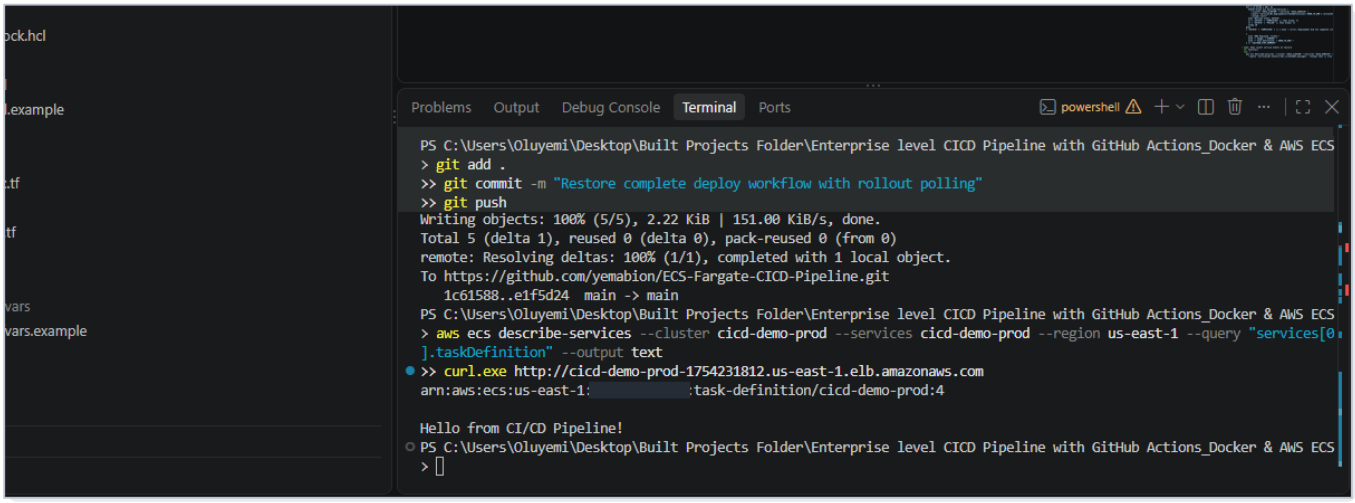


Figure 26. Terminal confirmation: the final commit is pushed, the service runs task definition **:4**, and the load balancer still answers **Hello from CI/CD Pipeline!**

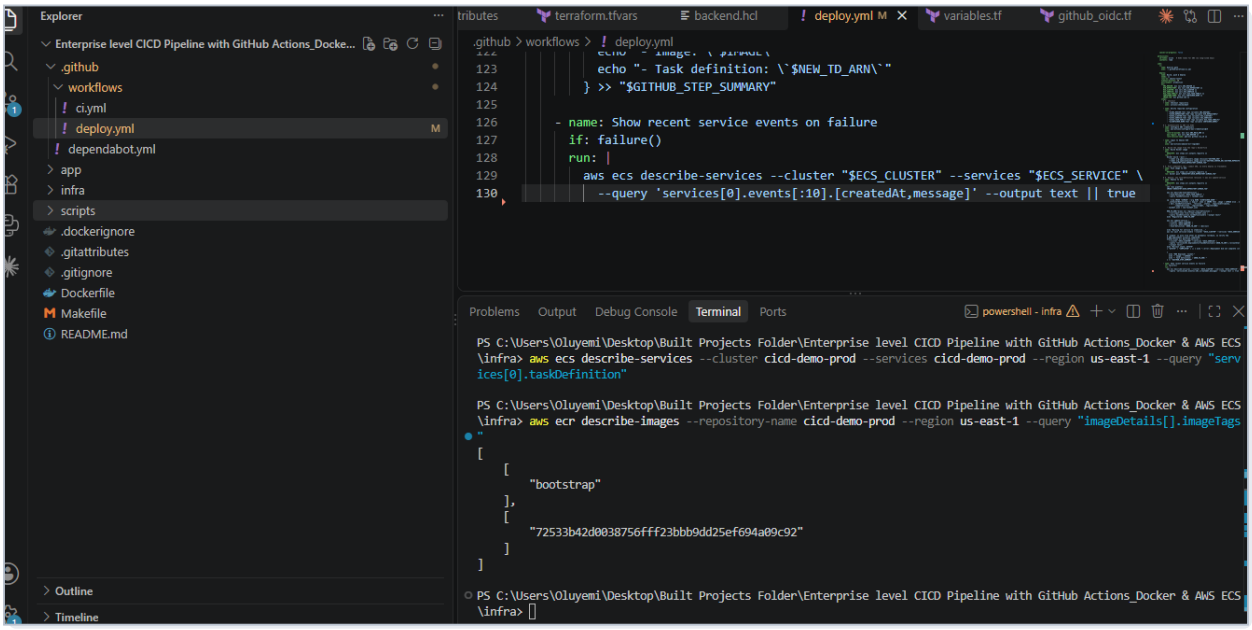


Figure 27. CLI view of the registry: the `bootstrap` image plus an image tagged with the commit SHA.

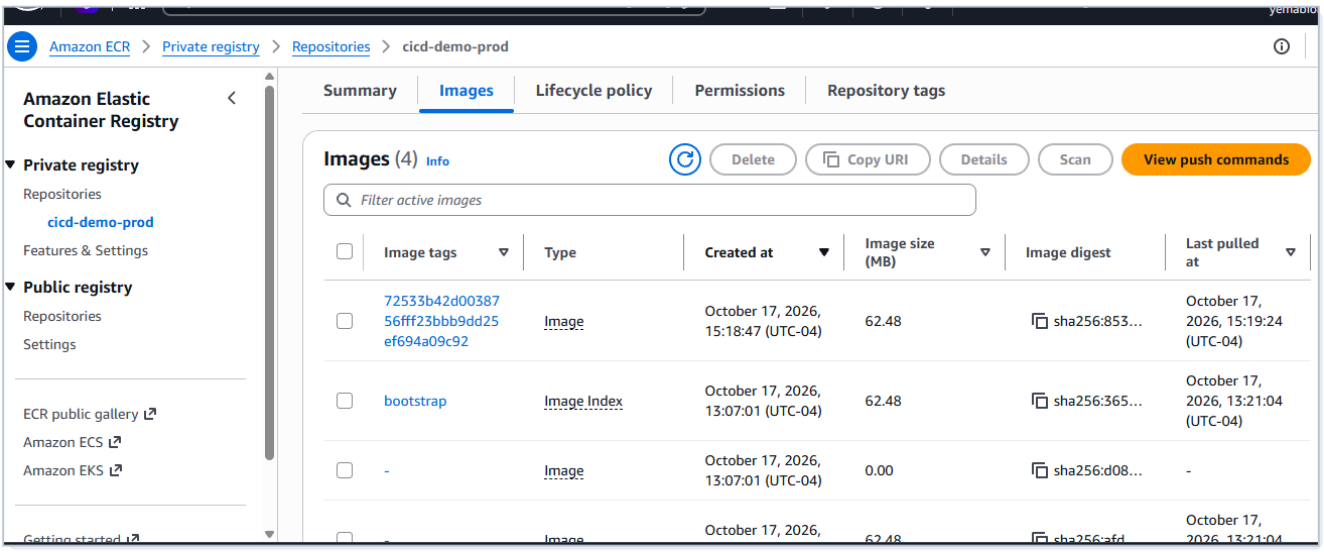


Figure 28. ECR after several pipeline runs: the bootstrap image and the SHA-tagged images the pipeline built.

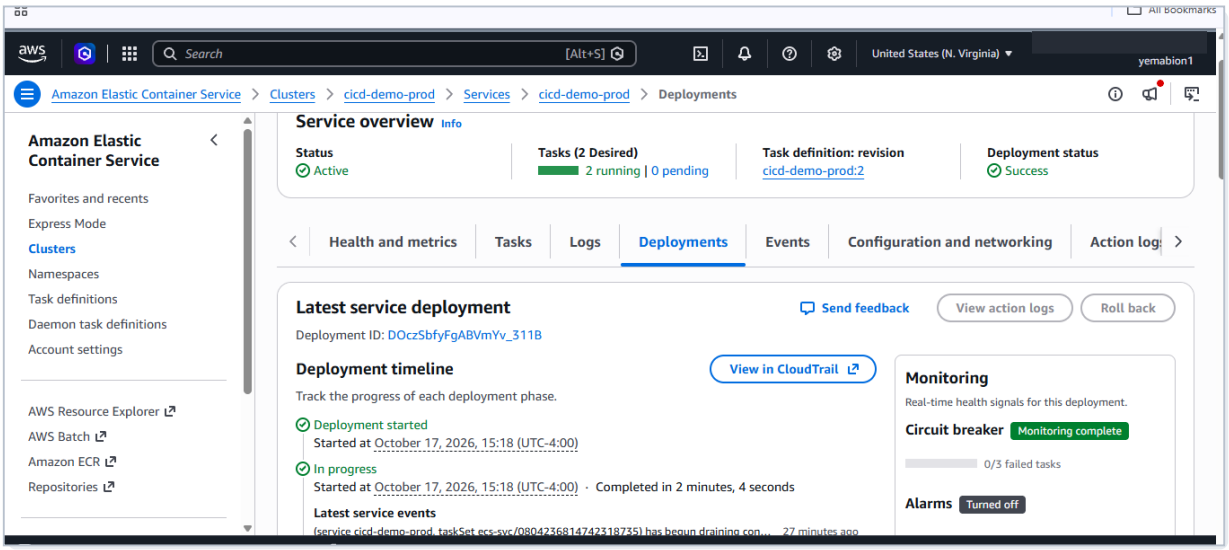


Figure 29. ECS *Deployments* tab: the latest deployment succeeded and the circuit breaker finished monitoring with **0 / 3 failed tasks**.

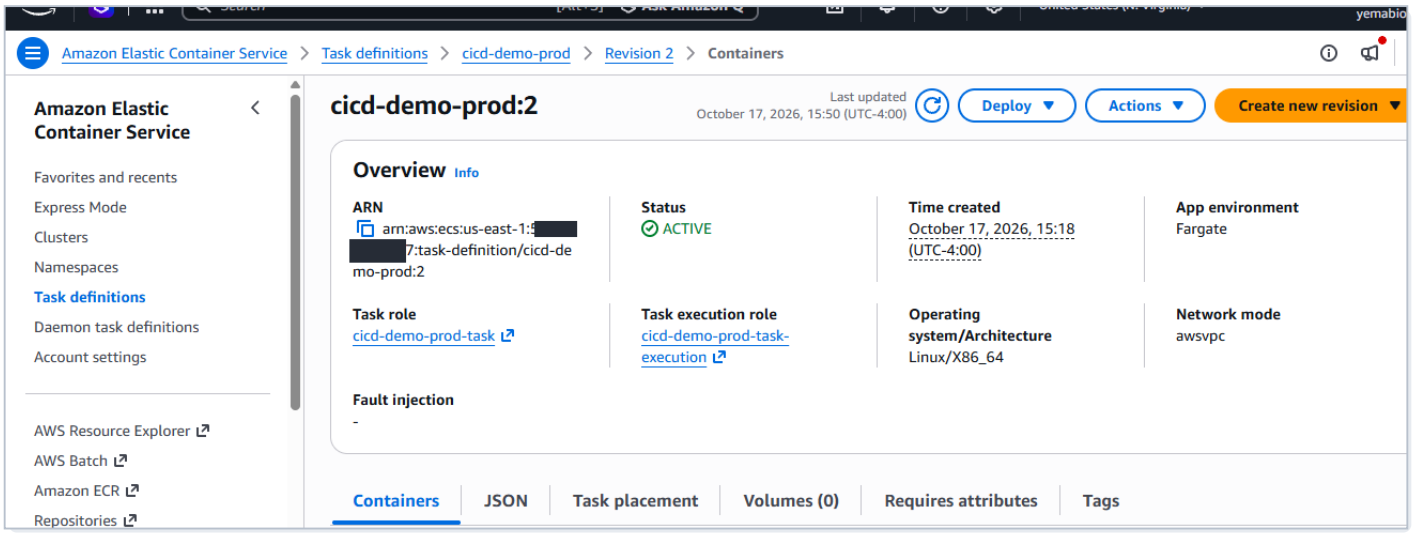


Figure 30. Task definition `cicd-demo-prod:2`, registered automatically by the pipeline.

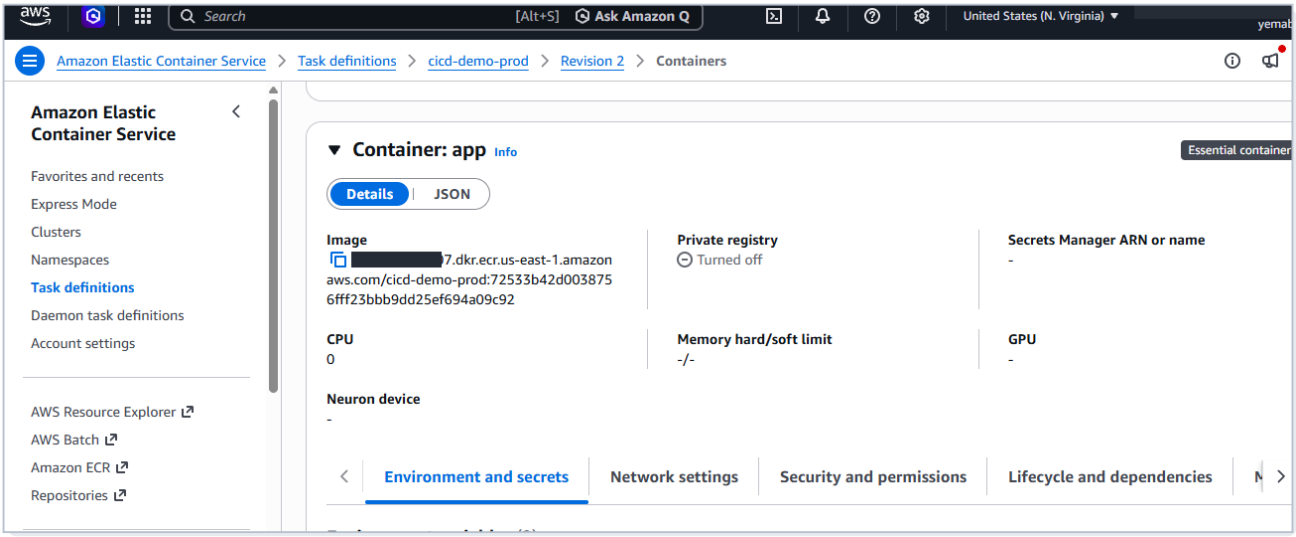


Figure 31. The container image in that revision is tagged with the **commit SHA**: full traceability from commit to running container.

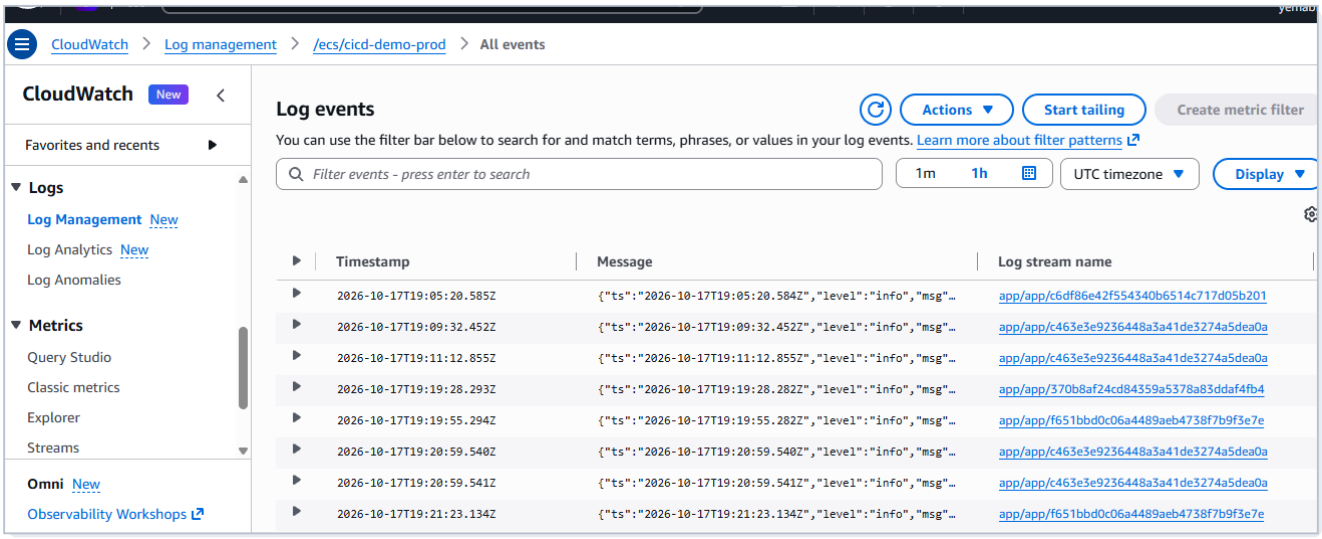


Figure 32. CloudWatch Logs: structured JSON logs streaming from both tasks.

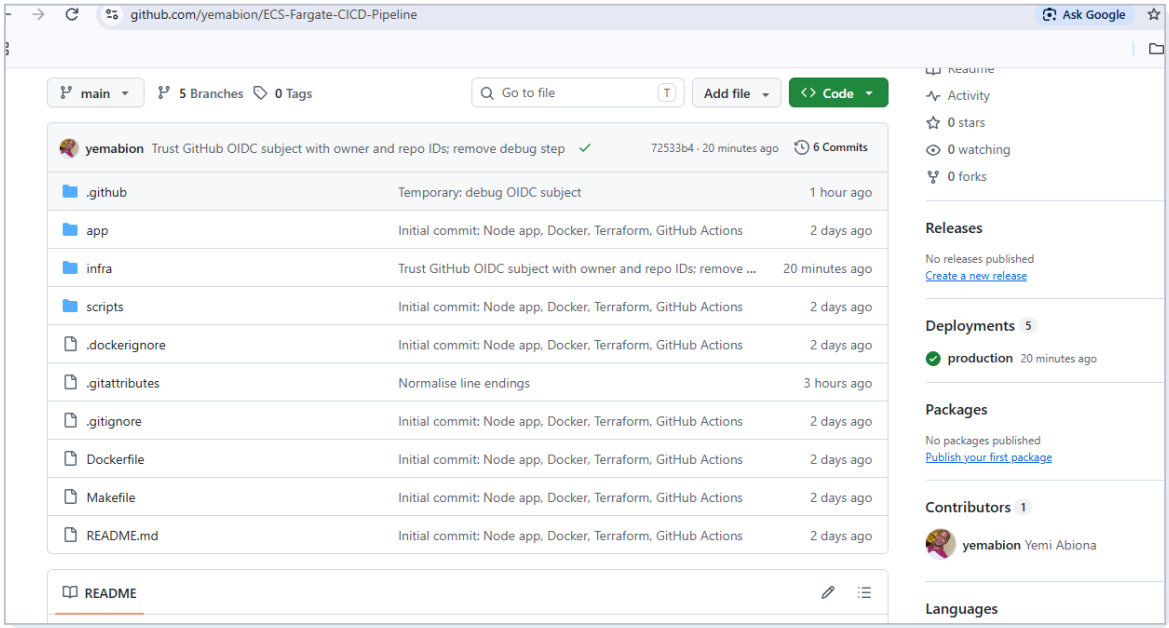


Figure 33. Repository structure: app, `infra` (Terraform), `scripts`, `Dockerfile`, `Makefile`, README and `.github/workflows`.

8 Lessons learned & next steps

What I would add for production

Key takeaways. Use OIDC instead of stored keys, and read the actual token claims when a trust policy fails. Immutable tags make releases traceable at the cost of re-deploying the same SHA. Verify the *deployment*, not just the service, and expect platform state to lag behind the CLI.

HTTPS	Add a domain and ACM certificate, an HTTPS listener on 443, and redirect 80 to 443.
Private networking	Move tasks to private subnets with VPC endpoints (ECR, S3, Logs) and drop their public IPs.
Edge protection	Add AWS WAF and ALB access logs; enable deletion protection.
Safer releases	Skip the push when the tag already exists; add smoke tests against the new tasks before shifting traffic.
Pull-request plans	Run `terraform plan` on pull requests with a read-only role.
Supply chain	Pin GitHub Actions and the base image by digest; add image signing.

Source code, Terraform and workflows: `github.com/yemabion/ECS-Fargate-CICD-Pipeline`